PROJECT REPORT | 3 NOVEMBER 2026

AI-Powered Data Engineering Platform

A cloud pipeline that turns raw food-delivery files into trusted business metrics, then adds AI that reads customer reviews and answers questions in plain English.

Prepared by Abhishek Bhatt

Dataset: Zomato-style food-delivery data

S3 · Snowflake · dbt · Airflow · Python · Gemini · Streamlit

7 source tables

7 staging views

10 dbt mart models

16 automated data tests

4 scheduled tasks

3 AI features

1 EXECUTIVE SUMMARY

From raw files to answers anyone can ask for

Food-delivery businesses collect millions of order rows and thousands of free-text reviews. Analysts need numbers they can trust, and managers want answers without learning SQL. This project builds the whole path in one place: files land in cloud storage, load into a data warehouse, are cleaned and modelled into business tables, refresh automatically every day, and are then opened up to AI.

The AI layer does three things. It labels each review with a sentiment and a topic. It powers a chatbot that answers questions using the reviews it finds most relevant. And it turns an English question into a database query, shows the query, and runs it.

How to read this report

This is a working prototype that runs on a development setup. It has not been hardened for production, and this report does not claim production performance, row counts or run times, because none were recorded. Section 11 lists exactly what is built, what has gaps and what is still to do.

2 PROBLEM AND GOALS

What the platform is meant to solve

BUSINESS NEED	WHAT THE PLATFORM DOES ABOUT IT
Keep raw files safe and cheap to store	Amazon S3 holds the source files, one folder per table.
Load millions of rows reliably	Snowflake reads S3 directly and bulk-loads seven raw tables.
Get clean, consistent numbers	dbt cleans the raw data and builds revenue, delivery-time and restaurant tables, with automatic tests.
Refresh without manual work	Apache Airflow runs the load, the transformations and the AI step in order, daily.
Understand what customers write	Gemini labels reviews with sentiment, score, topic and the main issue.
Let non-technical people ask questions	A review chatbot and an English-to-SQL app, both in Streamlit.

3 SOLUTION OVERVIEW

How the pieces fit together

Think of the platform as a restaurant kitchen. Ingredients arrive at the receiving dock, go to the pantry, get prepared, are plated, and reach the customer. Each stage of the pipeline has a matching job.

RECEIVING DOCK

Amazon S3

Raw files arrive and wait, untouched.

PANTRY

Snowflake RAW

An exact copy of every file, stored as tables.

PREP STATION

STAGING views

Wash, trim and label: fix types, names and messy values.

PLATED DISHES

MARTS tables

Ready for the customer: revenue, delivery speed, rankings.

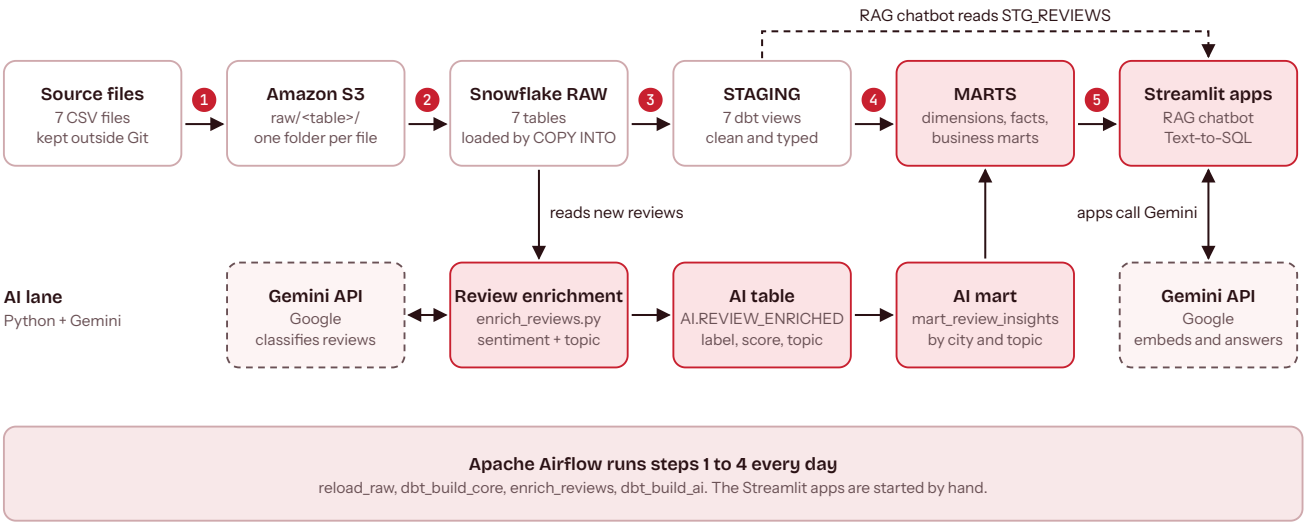


Figure 1. Source files travel left to right. Numbered arrows are explained in the table below. The lower lanes show where AI is added and how Airflow schedules the batch.

ARROW	WHAT HAPPENS	TOOL
1	Seven CSV files are uploaded into folders under `raw/` in the bucket. This step is manual today.	AWS S3
2	Snowflake reads the folders through a secure link and loads them into seven RAW tables.	Storage integration, `COPY INTO`
3	Seven views clean and type the raw columns. A view stores no data, it is a saved query.	dbt
4	Dimensions, two incremental fact tables and the business marts are built from the views.	dbt
5	The apps read the marts (Text-to-SQL) and the review view (chatbot).	Streamlit

4 SECURE STORAGE LINK

How Snowflake reads S3 without a password

Storing cloud keys inside a tool is a common way for credentials to leak. This project avoids it. Snowflake gets its own identity inside AWS, and a role in the project's AWS account says which identity is allowed to borrow read access, and only when it presents a shared secret called an External ID.

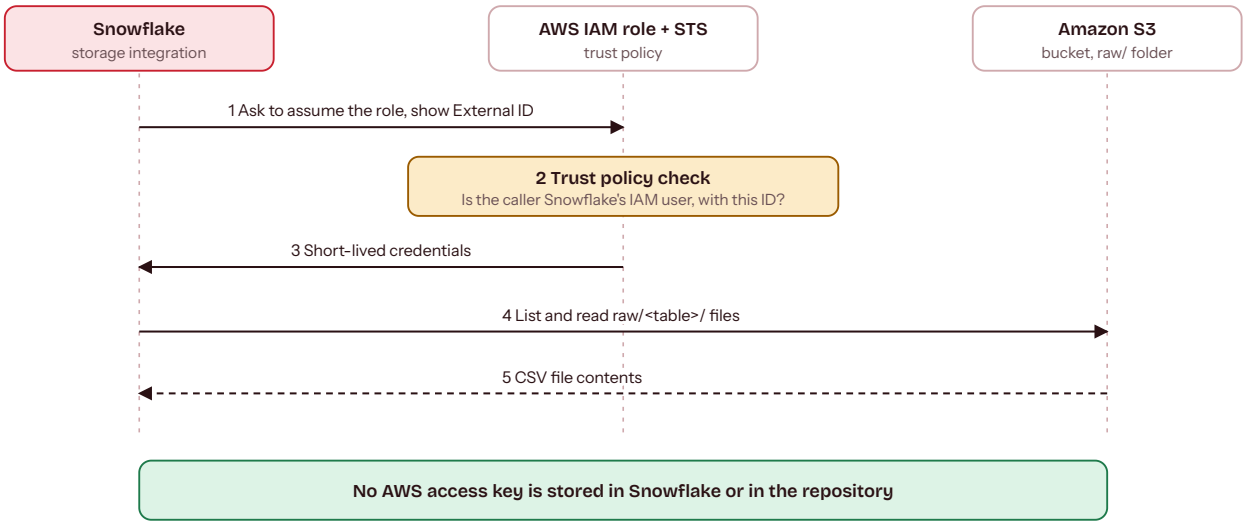


Figure 2. Snowflake holds a trust agreement, not a password. Each read starts with a fresh request for temporary access, which AWS grants only if the role's trust policy matches.

Setup order, and why order matters

1. Create a read-only permissions policy for the bucket (list and read only).
2. Create an IAM role with that policy and a temporary placeholder trust setting.
3. In Snowflake, create the storage integration `ZOMATO_S3_INT` and give it the role's address.
4. Run `DESC INTEGRATION` to get Snowflake's IAM user and the External ID.
5. Edit the role's trust policy to trust exactly that user, with exactly that External ID.
6. Validate the link, create the external stage `ZOMATO_RAW_STAGE` and the CSV file format `CSV_FMT`, then `LIST` the stage and run `COPY INTO`.

A lesson learned

Re-creating the integration produces a new External ID, which breaks the trust policy again. Once the link works, it should be changed with `ALTER`, not re-created.

5 WAREHOUSE DESIGN

One database, four working schemas

Everything lives in the Snowflake database `ZOMATO`, processed by an extra-small warehouse `ZOMATO_WH` that suspends itself after 60 seconds of inactivity to keep costs low.

SCHEMA	ROLE IN THE PIPELINE	WHAT IT HOLDS
RAW	Ingestion	Seven tables, a copy of the source files. Messy dimension columns are kept as text on purpose, so loading never fails on odd values.
STAGING	Cleaning	Seven dbt views that rename, convert and filter.
MARTS	Business-ready data	Four dimensions, two incremental fact tables, three business marts and one AI mart.
AI	Enrichment	One table, `REVIEW_ENRICHED`, written by the Python script.

Two further schemas, BRONZE and SNAPSHOTS, are created by the setup script but not used by any code yet.

How loading behaves

- Rows are matched to columns by position, so each table's column order mirrors its CSV.
- The four dimension files are real, messy data, so bad rows are skipped and loading continues. The three fact files are clean and generated, so any error stops the load and counts stay exact.
- Snowflake remembers which files were already loaded (for 64 days) and skips them. The daily run therefore picks up only new files instead of duplicating rows.

6 TRANSFORMATIONS

dbt turns raw tables into business tables

dbt lets each transformation be a plain SQL file. It works out the build order from references between files, runs the SQL inside Snowflake, and runs data tests after each build. That keeps the logic in version control and repeatable.

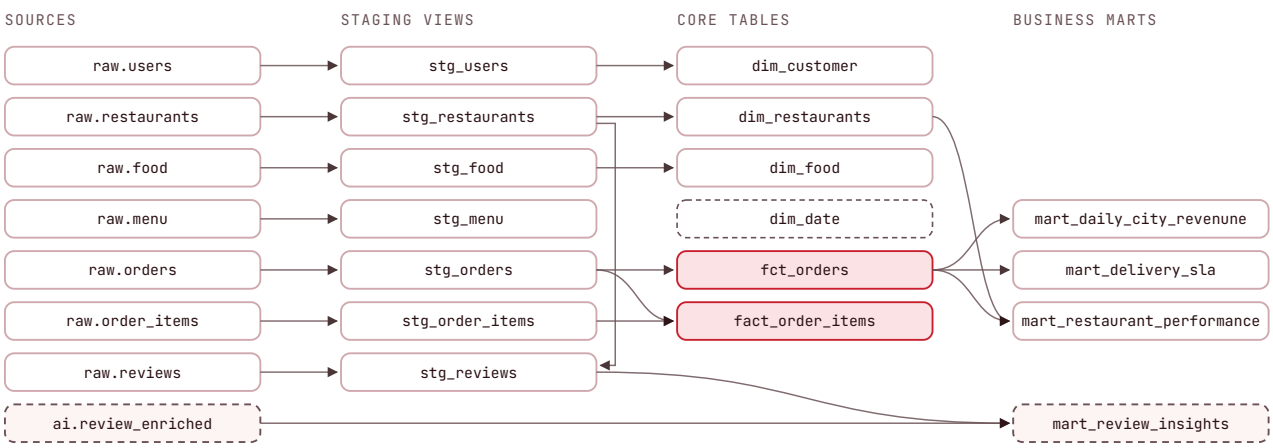


Figure 3. Each line means one model reads another. Staging views clean the raw tables, core tables hold the reusable facts and dimensions, and business marts answer specific questions. Shaded boxes are the two incremental tables. Dashed grey outlines mark the AI source and AI mart. dim_date is generated without a source table and no model reads it yet.

What the cleaning does

- **Restaurants:** turns `--` ratings into empty values, pulls numbers out of text such as rating counts and cost for two, and takes the city from after the last comma in the address text.
- **Users:** lower-cases emails, converts age and family size to numbers, and leaves the password column out.
- **Orders:** renames keys to `customer_id` and `restaurant_id`, cleans the city, and adds an `is_delivered` flag.
- **Reviews:** keeps reviews that have text and adds the restaurant's city.

The business tables

TABLE	QUESTION IT ANSWERS	HOW IT IS CALCULATED
mart_daily_city_revenune	How is each city selling, day by day?	Orders, delivered orders, cancel rate (cancelled / all orders), GMV (sales amount of delivered orders) and AOV (GMV / delivered orders).
mart_delivery_sla	How fast do we deliver?	Median and 90th-percentile delivery minutes for delivered orders, by city and hour of day.
mart_restaurant_performance	Which restaurants perform best?	Orders, delivered revenue, average rating and average delivery minutes per restaurant.
mart_review_insights	What do customers say, and where?	Review counts, average sentiment, average stars and flagged issues by city, topic and sentiment.
dim_customer	Who are the customers?	Adds an age segment: under 25, under 40, under 55, 55 and over, or unknown.

The table name `mart_daily_city_revenune` keeps a spelling slip from the original code. It is listed as it exists in the warehouse.

Incremental facts

The orders table is large, so `fct_orders` and `fact_order_items` do not rebuild every day. After the first build, each run looks only at orders newer than the latest one already stored and merges them in by key. A known limit is that an order arriving late with an older timestamp would be missed, and a look-back window is on the roadmap.

Core first, AI last

The AI mart needs a table that the Python step creates. The build is therefore split with a dbt tag: everything except the tag runs first, then the enrichment script runs, then only the tagged model is built. One failing AI call cannot block the core tables.

7 DAILY AUTOMATION

Airflow runs the pipeline in order

Without a scheduler, someone must remember to run each step, in the right order, and notice when one breaks. Airflow does the remembering. A workflow definition, called a DAG, lists the tasks and their order. Airflow runs them on schedule, keeps a log for each task and shows the history on screen.

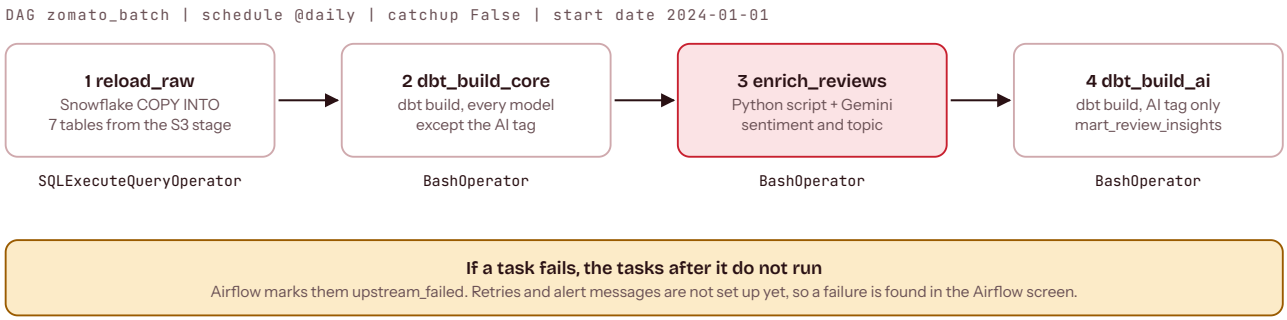


Figure 4. The four tasks always run in this order. The AI mart is built last because it reads the table that task 3 creates.

How it runs on a laptop

CONTAINER	JOB
postgres	Airflow's own memory: run history and task states. It does not hold project data.
airflow-init	Runs once to prepare the database and create the admin user.
apiserver	The web screen on port 8080 and the interface tasks report to.
scheduler	Decides what is due and runs the tasks.
dag-processor	Reads the workflow files and keeps them up to date.

The project folders for dbt and the AI script are mounted into the containers. dbt is installed in its own virtual environment inside the image so its packages never clash with Airflow's.

8 AI FEATURES

Three ways Gemini is used

8.1 Review enrichment: turning text into columns

Each review is sent to Gemini with strict instructions to answer in JSON. The answer must contain a sentiment (positive, negative or neutral), a score from -1 to 1, one of six topics (food quality, delivery, pricing, service, packaging, other) and a short main issue of six words or fewer. The script checks the answer against those rules and only then saves it. It handles only reviews that are not yet in the output table, so each review is paid for once.

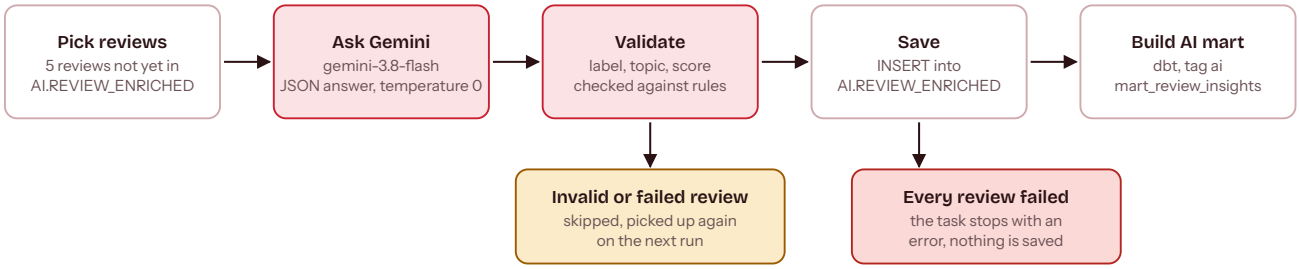


Figure 5. Allowed values are positive, negative or neutral for sentiment, a score from -1 to 1, and one of six topics. Anything else is rejected before it reaches the warehouse.

8.2 RAG chatbot: answers grounded in real reviews

A general AI model has never seen this company's reviews and would guess. Retrieval-augmented generation fixes that by finding relevant reviews first. Every review is converted to a list of numbers (an embedding) that captures its meaning. A question is converted the same way, the five closest reviews are found by cosine similarity, and Gemini is told to answer only from those five. The reviews used are shown under the answer so a reader can check them.

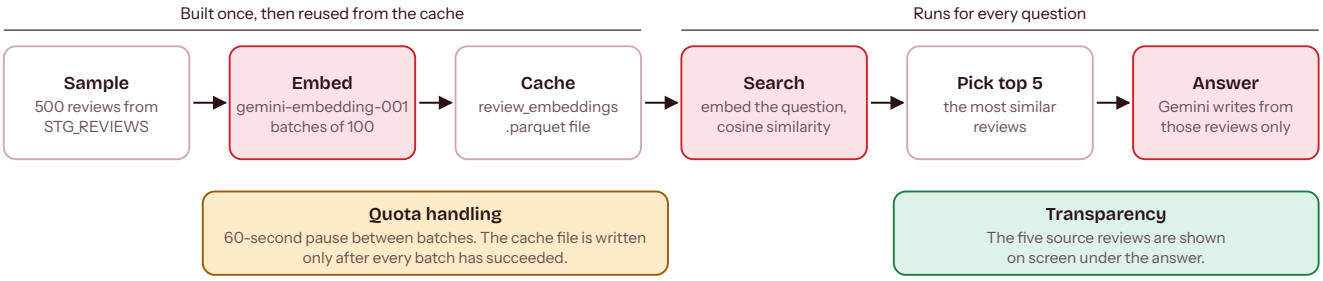


Figure 6. The corpus is a 500-review sample held in a local file. It is not a vector database, which is fine at this size and the first thing to replace at scale.

8.3 Text-to-SQL: ask the warehouse in English

The user types a question such as "Top 10 cities by GMV". Gemini receives the list of tables and columns and returns one query. The app shows the query, checks it, runs it in Snowflake, and displays a table, plus a bar chart when the result has two columns and the second is numeric.

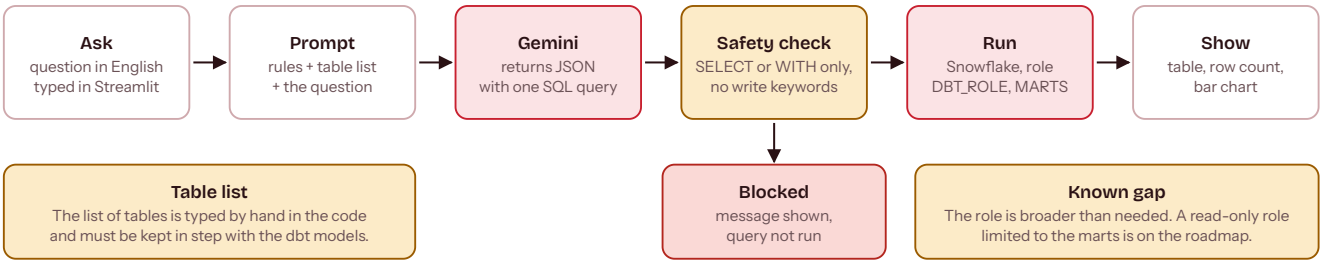


Figure 7. The generated SQL is displayed before it runs, so a reviewer can read what was asked of the warehouse.

Two gaps to fix before wider use

The safety check is a pattern match, which is a useful first filter but not a security boundary. Database permissions are the real protection, and the app currently runs with a broader role than it needs. Also, the table list given to Gemini was typed by hand and differs from the real tables in several places (for example the restaurant table is `DIM_RESTAURANTS`, and the delivery table has `p50` and `p90` columns), so some questions will fail until it is generated from the dbt models.

9 DATA QUALITY

What is checked automatically

Sixteen dbt tests run as part of every `dbt build`.

CHECK	WHERE
Unique and not empty	order id, order item id, restaurant id, customer id (users), review id, and order id in the orders fact
Not empty	customer id and restaurant id on staged orders
Relationship	every order's customer exists in the customer dimension
Accepted values	order status is Delivered, Cancelled or Refunded
AI output rules	sentiment, topic and score validated in Python before saving

Not yet in place: a row-count reconciliation between layers, tests on the other marts and the AI table, and freshness checks on the source files. Example queries for these checks are in the project documentation.

10 SECURITY

In place

- No cloud keys: access uses an IAM role with an External ID.
- Secrets live in environment files and a dbt profile that Git ignores; a scan of the repository found no keys.
- The AI key is read from the environment, never from code.
- The password column is not carried into the cleaned user view.

Still to do

- Give Text-to-SQL a read-only role limited to the marts, with a query timeout.
- Mask customer emails and avoid loading sensitive columns at all.
- Replace local development defaults (admin login, empty encryption key, placeholder secret) before any shared use.

11 IMPLEMENTATION STATUS

What exists, what has gaps, what does not

Built

- S3 to Snowflake secure link, stage and file format
- Seven RAW tables and the COPY loads
- Seven staging views, ten mart models, 16 tests
- Two incremental fact tables
- Airflow DAG in Docker with four tasks
- Review enrichment with output validation
- RAG chatbot with embedding cache

Built, with gaps

- Text-to-SQL: hand-typed table list, broad database role
- Airflow AI task: the container image still lists the old AI package and lacks the Gemini ones, so this task needs confirming
- Rate-limit handling is a fixed pause, with no automatic retry
- The full four-task Airflow run is not yet recorded as evidence

Not built

- Task retries and failure alerts
- Continuous loading with Snowpipe
- Vector database for the chatbot
- History tracking (SCD2 snapshots)
- Row-count reconciliation tests
- CI checks on pull requests, BI dashboard

12 PROBLEMS SOLVED

Issues met while building, and how they were handled

ISSUE	CAUSE	FIX	STATUS
AWS refused the role (`AssumeRole`)	The role still trusted a placeholder instead of Snowflake's IAM user and External ID	Copied both values from `DESC INTEGRATION` into the trust policy	Resolved
Streamlit warnings about a missing runtime	The app was started with `python`	Start with `streamlit run`	Resolved
Moving from OpenAI to Gemini	Different package, client and method names; calling the module like a function	Rewrote the three scripts for the Google GenAI client	Partly resolved
Embedding request rejected (500 texts)	Limit of 100 texts per request	Send in batches of 100	Resolved
429 quota exceeded on the free tier	Too many embedding calls per minute	60-second pause between batches, smaller test samples	Partly resolved
Airflow: `User is empty`	The container had no Snowflake user; the host shell's variables do not reach containers	Provided the variables to Docker Compose and recreated the services	Needs check
Empty encryption key warning in Airflow	Local development default	Documented; a real key is required beyond a laptop	Open
GitHub rejected the password	Passwords are no longer accepted for Git over HTTPS	Browser sign-in through Git Credential Manager	Resolved

A review of the code also found the table-list mismatch in Text-to-SQL, the missing Gemini packages in the Airflow image, and a hard-coded batch size of five reviews in the enrichment script. These are recorded in the troubleshooting guide as open items.

13 ROADMAP

Next steps, in priority order

PRIORITY	IMPROVEMENT	EFFORT
1	Read-only Snowflake role, table allow-list, query limit and timeout for Text-to-SQL	Low
1	Generate the Text-to-SQL table list from the dbt models	Low
1	Add Gemini packages and settings to the Airflow image; read the batch size from configuration	Low
2	Task retries and failure alerts; proper secrets and encryption key	Low to medium
2	More dbt tests: row-count reconciliation, source freshness, marts and AI table	Medium
3	Vector store and per-batch embedding checkpoints for the chatbot	Medium
3	Snowpipe for continuous loading; CI that runs dbt on pull requests	Medium
4	Mask personal data; add a dashboard on the marts	Medium

APPENDIX

Reference

Technology stack

LAYER	TECHNOLOGY
Storage	Amazon S3, IAM role, Snowflake storage integration
Warehouse	Snowflake (database ZOMATO, warehouse ZOMATO_WH)
Transformation	dbt with the Snowflake adapter, version 1.8
Orchestration	Apache Airflow 3.0.3 on Docker Compose with Postgres
Processing	Python, pandas, numpy, Snowflake connector
AI	Google Gemini API: gemini-3.8-flash and gemini-embedding-001
Interface	Streamlit

Glossary

TERM	MEANING
GMV	Gross merchandise value: here, the sales amount of delivered orders.
AOV	Average order value: GMV divided by delivered orders.
Stage (Snowflake)	A named pointer to a storage folder that Snowflake can load from.
View / table	A saved query that stores nothing / stored rows.
Incremental model	A table that adds only new rows on each run.
DAG	A workflow of tasks with a fixed order and no loops.
Embedding	A list of numbers that represents the meaning of a text.
RAG	Retrieval-augmented generation: find relevant text first, then answer from it.

Source code and documentation

Repository: github.com/abhishekbhattt/AI-Powered-Data-Engineering-Platform. The repository documentation covers architecture, a full setup guide, a chronological troubleshooting log and interview notes.

Prepared by Abhishek Bhatt, 3 November 2026. This is an independent learning project built on a Zomato-style food-delivery dataset. It is not affiliated with, produced by or endorsed by Zomato. Figures such as row counts, run times and costs are not reported because none were recorded. Credentials, account identifiers and keys are intentionally not included.